

Senior Android Engineering Knowledge Base для self-assessment в 2026

Рамка и критерии

Эта база знаний калибрована не под “знает API”, а под “умеет принимать архитектурные и эксплуатационные решения в production”. За основу взят современный Android-стек, который сам Android team описывает через unidirectional data flow, state holders, coroutines/Flow, модульность и performance-first подход. При этом я **осознанно** держу фокус на вашем целевом наборе: Jetpack Compose как основной UI, Coroutines/Flow, MVI state machine, Dagger 2 с ручной сборкой графа, Room + DataStore, Retrofit + OkHttp, custom Fragment-based back stack, JUnit5 + MockK + Compose UI testing, Version Catalogs, R8, Baseline Profiles и modularization. Официальная документация Android в 2026 году действительно рекомендует Hilt для DI и Compose-first navigation стек вроде Navigation 3 для Compose-only приложений, а Interop-гайд прямо называет Fragment + ComposeView скорее переходной или hybrid-стратегией. Но для крупных кодовых баз с кастомной навигацией, feature-графами и постепенной миграцией именно такой стек по-прежнему реалистичен и требует senior-level глубины. ¹

Ниже каждый раздел устроен как interview-calibration: короткая theory block, затем вопросы, которые отличают surface-level familiarity от системного engineering мышления. “Senior answer” здесь не означает “самый длинный”; это ответ, который правильно ставит границы, проговаривает trade-offs, связывает локальное решение с жизненным циклом экрана, back stack, тестируемостью, деградацией производительности и стоимостью поддержки. ²

Concurrency и reactive state

Kotlin и coroutines internals

Theory block.

Senior Android-разработчик в 2026 должен мыслить не “где launch”, а “кто владеет scope, кто владеет dispatcher, и как failure/cancellation распространяются по дереву job-ов”. Structured concurrency означает, что дочерние корутины живут внутри родительского scope и влияют на завершение родителя; исключение в child по умолчанию отменяет siblings и parent, если не используется supervision. `CoroutineExceptionHandler` — это не бизнес-обработка ошибок, а последний рубеж для **uncaught** исключений корневой корутины; он не “чинит” обычный control flow и не перехватывает всё подряд. Cancellation кооперативна: она проявляется в suspension points и должна уважаться в CPU-bound коде и cleanup-блоках. На Android важны main-safety и явное владение dispatcher-ами: код data/domain слоя не должен случайно блокировать Main, а hardcoded dispatchers ухудшают тестируемость и контроль исполнения. Отдельный маркер зрелости — почти рефлекторное избегание `GlobalScope`: Android-документация прямо связывает lifecycle-tied scope с высвобождением ресурсов и отсутствием утечек. ³

Вопрос

Когда нужен `supervisorScope`, а когда обычный `coroutineScope`?

Junior answer.

`coroutineScope` отменяет все дочерние корутины при ошибке одной из них, а `supervisorScope` — нет. Поэтому `supervisorScope` нужен, когда задачи независимы. ⁴

Middle answer.

Если экран грузит несколько независимых блоков данных, `supervisorScope` позволяет показать частичный успех и локально обработать ошибку одного запроса. Если же дочерние задачи вместе формируют один атомарный результат, правильнее `coroutineScope`, чтобы не получить полусобранное состояние. ⁵

Senior answer.

Выбор определяется не “независимы/не независимы” сам по себе, а контрактом уровня state machine. Если MVI-state допускает деградацию по частям — supervision уместен; если reducer ожидает консистентный агрегат, supervision превращает failure semantics в неявную бизнес-логику и усложняет восстановление. Senior ещё проговаривает, кто собирает ошибки: actor, orchestration use case или слой repository, и не смешивает supervision с бесконтрольным проглатыванием ошибок. ⁶

Why this question matters.

Вопрос проверяет, понимает ли человек корутины как дерево жизненных циклов и failure propagation, а не как “асинхронные функции”. В production это напрямую влияет на частичный рендеринг, retry, пустые экраны, race conditions и предсказуемость state transitions. ⁷

Вопрос

Почему `CoroutineExceptionHandler` почти никогда не является основным механизмом обработки ошибок на экране?

Junior answer.

Потому что он вызывается не всегда и лучше использовать `try/catch`. Обычно ошибки надо ловить рядом с кодом, который их ожидает. ⁸

Middle answer.

Он подходит для логирования и crash reporting корневых coroutine failures, но не для нормальной бизнес-обработки. Исключения из `async` приходят в `await`, а child-корутины в structured concurrency часто завершают parent до того, как handler вообще становится релевантным. ⁸

Senior answer.

На senior-уровне ошибка — часть контракта use case или actor-a, а не факт существования глобального handler-a. Если экрану нужно перейти в `Error`, показать `News`, откатить optimistic state или решить, retry-able ли сбой, это должна делать state machine, а `CoroutineExceptionHandler` остаётся только страховочной сеткой для дефектов, которые уже вышли за пределы нормального control flow. ⁹

Why this question matters.

Такой вопрос быстро отделяет людей, которые действительно строили fault model на экране, от тех, кто видел handler в документации и переоценил его роль. Ошибка управления здесь почти всегда превращается либо в silent failure, либо в неуправляемый crash/noise в логах. ¹⁰

Вопрос

Кто должен владеть dispatcher selection и почему `hardcode Dispatchers.IO` глубоко в коде часто плохая идея?

Junior answer.

Dispatcher нужен, чтобы выбрать поток. Если захардкодить его везде, код станет менее гибким.

11

Middle answer.

Инъектируемые dispatchers упрощают тесты и позволяют `runTest` контролировать исполнение. Кроме того, `main-safety` — это обязанность слоя, который делает blocking или expensive work, а не UI-состава, который не должен гадать, где что переключать.

12

Senior answer.

У senior developer dispatcher — часть boundary contract: domain/use case может быть main-safe без знания о платформе, repository может изолировать blocking I/O, а orchestration-слой — ограничивать concurrency policy. Hardcoded dispatcher внутри глубокой зависимости закрепляет execution policy в неправильном месте, затрудняет deterministic tests, скрывает производственные bottleneck-ы и мешает централизованно менять scheduling under load.

13

Why this question matters.

Вопрос проверяет, понимает ли человек coroutines как средство управления исполнением системы, а не просто как замену callback-ам. В больших приложениях это влияет на reproducibility тестов, responsiveness UI и расследование перформанс-проблем.

14

Flow, StateFlow и SharedFlow

Theory block.

Зрелое понимание Flow начинается с выбора **семантики потока**, а не типа “который удобнее”. Обычный `Flow` обычно cold: каждый collector снова запускает upstream. `SharedFlow` и `StateFlow` hot: их активная инстанция живёт независимо от collectors. `StateFlow` — специализированный `SharedFlow` для состояния: он всегда имеет текущее значение, транслирует только последнее состояние и conflates emissions через `Any.equals`. `SharedFlow` лучше подходит для событий и broadcast-сигналов, где важны `replay`, buffer semantics и отсутствие “обязательного current value”. `stateIn` и `shareIn` создают sharing coroutine в заданном scope; реальная стоимость и поведение зависят от `SharingStarted`, `replay` и buffering. При миграции с RxJava важно не переносить механически категории Observable/Subject в Flow-мир: cancellation, buffering и подписка/подписчик-семантика отличаются. Для тестирования senior должен уверенно различать тесты “flow как input” и “flow как output”, использовать fake producers, continuous collection и `runTest`, а не надеяться, что hot stream “как-нибудь успеет эмитнуть”.

15

Вопрос

Когда UI state должен быть `StateFlow`, а когда нужен `SharedFlow`?

Junior answer.

`StateFlow` — для состояния экрана, `SharedFlow` — для событий. Например, список данных — `StateFlow`, snackbar — `SharedFlow`.

16

Middle answer.

Если новый подписчик должен немедленно увидеть текущее значение — это `StateFlow`. Если повторная доставка старого значения опасна или бессмысленна, как у `navigation/snackbar/result events`, нужен отдельный `SharedFlow` или `News`-канал без хранения события в `state`. ¹⁷

Senior answer.

На senior-уровне вопрос формулируется как “какова replay-семантика этого доменного факта?”. UI `state` обязан быть `idempotent` и восстанавливаемым, поэтому он живёт в `StateFlow`; одноразовые эффекты не являются частью `durable state` и должны жить отдельно, иначе `back stack`, `ресабскрайб` и `process recreation` начинают переигрывать старые действия. ¹⁸

Why this question matters.

Это один из самых дешёвых способов проверить, видел ли человек реальные баги с повторным `navigation`, фантомными `snackbar` и “событиями в `state`”. Ответ показывает, есть ли у кандидата модель времени и повторной подписки, а не только знание названий классов. ¹⁹

Вопрос

Какие скрытые проблемы приносит бездумное использование `stateIn` и `shareIn`?

Junior answer.

Они делают `flow hot` и позволяют шарить `upstream`. Нужно правильно выбрать `scope` и `SharingStarted`. ²⁰

Middle answer.

Ошибка часто в том, что долговечный `scope` делает `seemingly screen-local stream` фактически `process-wide`, а `WhileSubscribed` с неподходящим `timeout` перезапускает дорогой `upstream` чаще, чем ожидалось. Ещё одна ловушка — `replay/buffer semantics`, из-за которых подписчик получает “неожиданно старое” значение. ²⁰

Senior answer.

Senior видит `stateIn/shareIn` как изменение жизненного цикла источника данных: вы не просто “кешируете поток”, вы вводите новый `hot-owner` с собственной `cancellation` и `failure model`. Неправильный `owner` ломает ресурсные контракты, размывает ответственность за кэш между `feature` и `repository` и может скрыто превратить `cheap collector` в держателя сети, БД-слушателя или другого дорогого источника. ²¹

Why this question matters.

В боевых приложениях именно тут рождаются утечки, внезапные перезапросы и “почему после ухода со `screen upstream` ещё жив?”. Вопрос проверяет `ownership thinking`, а не знание пары операторов. ²²

Вопрос

Чем миграция с `RxJava` на `Flow` отличается от механической замены типов?

Junior answer.

Есть адаптеры `asFlow`, `asObservable`, `asFlowable`, поэтому миграция возможна постепенно. Но модели всё равно разные. ²³

Middle answer.

В RxJava команды часто строятся вокруг Subject-ов и backpressure-типов, а во Flow нужно заново определить cold/hot ownership, cancellation и buffer policy. Если просто обернуть старые цепочки, можно сохранить старую сложность, но потерять предсказуемость structured concurrency и тестируемость. ²⁴

Senior answer.

Хорошая миграция — это пересборка boundary contracts: Single-подобные вещи становятся suspend, durable state становится StateFlow, transient events — SharedFlow / News, а interop-адаптеры остаются на границах. Senior не тащит Rx-мышление о “всё поток”, если семантически часть API должна стать обычной suspend-операцией с явной отменой, structured scope и simpler error surface. ²⁵

Why this question matters.

Этот вопрос вскрывает, мигрировал ли человек реально legacy-код, или знает только слова “Flow вместо Rx”. В production цена ошибки — смесь двух ментальных моделей, где никто уже не понимает ни ownership, ни доставку событий. ²⁶

Compose state, recomposition и performance

Theory block.

Senior-level Compose — это прежде всего понимание того, **почему** происходит recomposition и где именно читается state. Compose может скипать recomposition, если входы skippable и stable; стабильность определяется компилятором по правилам immutability/наблюдаемости, а не по ощущениям разработчика. В 2026 Android docs отдельно рекомендуют сначала **диагностировать** stability-проблемы, а уже потом чинить их — например, через strong skipping, аннотации стабильности, изменение структуры типов или перенос state reads в более дешёвую фазу, вроде draw. State hoisting остаётся базовой дисциплиной: состояние поднимается к lowest common ancestor и наружу экспортируется как immutable state + intents/events. Side-effect API (LaunchedEffect , DisposableEffect , rememberUpdatedState , produceState , snapshotFlow) нужны не “чтобы код сработал”, а чтобы удерживать lifecycle-awareness и UDF. Типичные senior-грабли: нестабильные DTO в hot composable tree, отсутствие stable keys в lazy list, чтение часто меняющегося state в composition вместо layout/draw, и неправильная интеграция ComposeView во Fragment без корректной disposal strategy. ²⁷

Вопрос

Когда нестабильный параметр — это реальная проблема, а когда его можно не трогать?

Junior answer.

Нестабильные параметры могут вызывать лишние recomposition, поэтому лучше делать классы stable. Но не всегда это критично. ²⁸

Middle answer.

Если composable не находится в горячем пути, вызывается редко или реальная трассировка не показывает лишней работы, “чинить стабильность” может быть преждевременной оптимизацией. Документация Compose прямо советует сначала диагностировать, а уже потом применять стабильность/strong skipping. ²⁹

Senior answer.

Senior смотрит не на annotation count, а на cost model: где recomposition происходит, можно ли её скипнуть, и что реально дорого — composition, layout, draw или side-effects вокруг. Иногда правильнее не делать тип “stable”, а разрезать tree, поменять state ownership, вынести derived values или сместить чтение state в lambda-based modifier, чтобы Compose обходил более тяжёлые фазы. ³⁰

Why this question matters.

Вопрос отличает людей, которые умеют профилировать Compose, от людей, которые видят любую нестабильность как дефект сами по себе. Это напрямую влияет на стоимость support-а: можно потратить недели на annotations и не починить реальный bottleneck. ³¹

Вопрос

Как выбирать между `LaunchedEffect`, `DisposableEffect` и `rememberUpdatedState`?

Junior answer.

`LaunchedEffect` запускает корутину, `DisposableEffect` нужен для подписки/очистки, `rememberUpdatedState` помогает не рестартовать эффект при обновлении callback-а. ³²

Middle answer.

Если нужна lifecycle-aware coroutine на время присутствия composable — `LaunchedEffect`. Если вы регистрируете listener, observer или callback и обязаны корректно отписаться — `DisposableEffect`. Если долгоживущий эффект должен видеть свежий lambda/reference без перезапуска, используйте `rememberUpdatedState`, иначе легко получить stale callback. ³²

Senior answer.

Senior ещё спрашивает: должен ли этот эффект вообще жить в Composition, или это уже responsibility state holder-а/MVI-feature. Эффекты в Compose не должны ломать UDF: навигация, snackbar, analytics и внешние подписки должны входить в UI как реакция на state/event contract, а не превращать composable в второй orchestration-layer. ³³

Why this question matters.

Здесь проверяется зрелость в работе с declarative UI: человек либо понимает Composition lifecycle, либо просто “подбирает API, пока работает”. Production-цена ошибок — дублирующиеся подписки, потерянные cleanup, stale callbacks и нестабильные тесты. ³⁴

Вопрос

Почему в lazy списках так важны keys и где часто прячется реальная perf-проблема?

Junior answer.

Keys помогают Compose правильно сопоставлять items при reorder/insert/remove. Без keys remembered state может поехать. ³⁵

Middle answer.

Настоящая проблема часто не только в самих keys, а в том, что item читает слишком много общего state или пересоздаёт нестабильные параметры на каждый frame. Тогда даже правильные keys не спасают от широких recomposition. ³⁶

Senior answer.

Senior думает о списке как о горячем контуре производительности: нужны stable item models, локализованные state reads, аккуратные remember/derivedStateOf и минимально возможные invalidation regions. Хороший ответ обычно связывает keys с сохранением item-local state, а perf — с тем, сколько узлов мы заставляем дойти до composition/layout/draw на каждое изменение. ³⁷

Why this question matters.

Списки — место, где surface-level Compose knowledge ломается быстрее всего. Ответ показывает, видел ли человек реальные проблемы скролла, partial updates и неожиданного state drift внутри item-ов. ³⁸

MVI и unidirectional data flow

Theory block.

Сильная MVI-модель на Android — это не просто “один `StateFlow<State>` на экран”. Суть в том, что state production идёт как pipeline: inputs → state holder/state machine → immutable UI state → render. В MVICore-подобной схеме `Actor` занимается асинхронной или условной работой и порождает `Effect`, `Reducer` детерминированно вычисляет новый `State`, а `NewsPublisher` обрабатывает одноразовые сигналы, которые бессмысленно хранить в state — например, redirect, snackbar, transient error. Это критично: MVICore прямо описывает `News` как решение recurring-проблемы “events that should be consumed only once” и рекомендует не класть такие вещи в state, чтобы новые подписчики не переигрывали старые события. Senior-понимание MVI — это ещё и чувство меры: MVI отлично масштабирует сложные экраны с параллельными источниками, retry, optimistic updates и явной replay-семантикой, но легко деградирует в boilerplate, если reducer становится мусоросборником, а actor — скрытым god object. Android-гайды по UDF здесь хорошо совпадают с MVI: immutable state наружу, intents внутрь, логика state production вне UI. ³⁹

Вопрос

Почему navigation/snackbar/result event не должны жить внутри screen state как флаг?

Junior answer.

Потому что state переиспользуется и событие может сработать повторно. Для одноразовых событий лучше отдельный канал. ⁴⁰

Middle answer.

Если положить redirect/snackbar в state, новый collector или возврат на экран может повторно получить уже устаревшее действие. MVICore отдельно выделяет `News` именно как способ держать single-consumption events отдельно от durable state. ⁴¹

Senior answer.

Senior отвечает через семантику времени: state должен быть воспроизводимым снимком экрана, а event — факт перехода/реакции, у которого другая replay policy. Смешение этих двух временных моделей обычно ломает back stack, process recreation и тесты, потому что “история того, что нужно сделать один раз” маскируется под “текущее состояние”. ⁴²

Why this question matters.

Это практически идеальный production-фильтр на зрелость MVI/UDF-мышления. Если человек

настраивает на `showSnackBar = true` в state без разговора о replay-семантике, почти наверняка он ещё не проживал реальные баги жизненного цикла. ⁴³

Вопрос

Где проходит граница между `Actor` и `Reducer` ?

Junior answer.

`Reducer` меняет state, `Actor` делает асинхронную работу. Обычно network и validation идут в `Actor`, а чистое преобразование state — в `Reducer`. ⁴⁴

Middle answer.

Хороший reducer должен быть максимально dumb и детерминированным: получил effect — выдал новый state. `Actor` инкапсулирует async jobs, conditional branching и взаимодействие с внешним миром, чтобы бизнес-ветвление не пряталось внутри state reduction. ⁴⁵

Senior answer.

Senior обычно обсуждает не только синтаксическую границу, но и эволюцию feature: если reducer начинает зависеть от времени, IO или внешних сервисов, вы теряете replayability и компактность reasoning. Если actor, наоборот, начинает тайно мутировать всё подряд и решать половину state machine, MVI теряет прозрачность и тестируемость; значит, границы надо сдвигать на уровне эффектов и action-модели, а не только классов. ⁴⁶

Why this question matters.

В production главная польза MVI — не “модное слово”, а возможность локально рассуждать о state transitions и error handling. Этот вопрос быстро показывает, понимает ли человек систему как state machine, или просто воспроизводит шаблон из прошлой codebase. ⁴⁷

Архитектура, DI и data layer

Dagger 2 с ручной сборкой графа

Theory block.

Senior-уровень в Dagger 2 — это способность проектировать граф зависимостей как часть архитектуры приложения, а не просто “собрать инъекцию”. Dagger остаётся compile-time DI framework, который генерирует код, близкий к hand-written wiring, и валидирует binding graph на этапе компиляции. Для Android-кодовой базы с custom navigation это особенно ценно: вы явно контролируете feature graph, scopes, lifetime объектов и границы модулей. `Subcomponent` наследует bindings родителя и удобен для вложенных scope-ов; multibinding позволяет собирать plugin-like registry без knowledge о конкретных реализациях в месте использования. В то же время merged key space subcomponent-ов может размыть policy boundaries, если их не подкреплять visibility/qualifier discipline. Assisted injection нужен там, где часть параметров приходит из runtime, но такие типы по правилам Dagger не скопируются и не могут быть scoped. Hilt в 2026 остаётся официально рекомендуемым на Android и упрощает Android-specific boilerplate через predefined components, однако платой становятся меньшая явность графа, большее следование framework conventions и не всегда удобные trade-offs в глубоких multi-module/custom-host архитектурах. ⁴⁸

Вопрос

Когда для feature-модуля вы предпочтёте `Subcomponent`, а когда `component dependency`?

Junior answer.

`Subcomponent` подходит для вложенного графа, а `component dependency` — когда графы больше изолированы. Оба варианта рабочие. ⁴⁹

Middle answer.

Если feature должен естественно наследовать родительские bindings и жить во вложенном scope, `subcomponent` обычно проще. Если нужна более жёсткая изоляция и явный `exported surface`, зависимости компонентов могут быть понятнее, хотя они сложнее для распространения `multibindings`. ⁴⁹

Senior answer.

Senior смотрит на организационную границу: кто “владеет” `binding key space` и насколько допустимо, чтобы `parent bindings` были доступны `child` по умолчанию. Если важнее дебаггинг и естественное наследование scope-ов — `subcomponent`; если важнее enforce-ить публичный контракт feature-модуля как библиотеки — `component dependency` может дать дисциплину ценой дополнительной сложности и `less ergonomic graph composition`. ⁵⁰

Why this question matters.

Вопрос проверяет, умеет ли человек связывать DI с модульностью и `ownership boundaries`. В крупных приложениях это влияет не только на кодогенерацию, но и на то, насколько команда способна локально изменять feature без пробоя по всему графу. ⁵¹

Вопрос

Чем ручной Dagger 2 может быть лучше Hilt именно в большой multi-module codebase с custom Fragment navigation?

Junior answer.

Hilt проще в настройке, а Dagger 2 даёт больше контроля. Поэтому в сложных проектах Dagger 2 иногда удобнее. ⁵²

Middle answer.

Ручной Dagger позволяет явно собирать feature graph под конкретный `back stack entry`, `Fragment host` или `runtime factory` и не подстраиваться под стандартные Android components Hilt. Это особенно полезно, когда приложение уже имеет собственную навигационную и scope-модель. ⁵³

Senior answer.

Senior не отвечает лозунгом “Dagger гибче”, а проговаривает стоимость владения: `manual Dagger` выигрывает там, где явный `graph ownership`, независимые `feature boundaries` и нестандартные `lifetime-модели` критичнее `boilerplate reduction`. Hilt выигрывает там, где стандартный Android `lifecycle` и `build ergonomics` дают больше пользы; но если архитектура уже декомпозирована вокруг собственных `entry points` и `scope-ов`, Hilt может навязать другой центр тяжести системы, а не просто “сократить код”. ⁵⁴

Why this question matters.

Такой вопрос отделяет людей, которые реально выбирали DI-стратегию под архитектуру, от тех, кто просто повторяет рекомендации по умолчанию. Для senior это не религиозный спор, а решение о стоимости дальнейшей эволюции кодовой базы. ⁵⁵

Room и DataStore

Theory block.

Старший Android-инженер должен видеть Room и DataStore как два разных инструмента с разной durability/shape-семантикой. Room — это слой над SQLite с compile-time проверкой SQL, поддержкой `suspend`-операций, observable queries через `Flow` и полноценными migration paths. Для production важны не “DAO работает”, а локальный source of truth, корректные transaction boundaries, предсказуемая миграция схемы и тестирование этих миграций до релиза. Room хорошо ложится на offline-first и на архитектуру, где UI пересобирается из локального состояния после process death. DataStore, напротив, рассчитан на небольшие объёмы данных, которые нужно хранить асинхронно, консистентно и транзакционно; Android docs прямо позиционируют его как замену `SharedPreferences` для новых сценариев. В 2026 рекомендации особенно подчёркивают: DataStore должен жить в data layer как singleton и не читаться/писаться прямо из composable. Практический senior-критерий здесь простой: transient UI state — не тащить в Room; сложные реляционные сущности — не запихивать в DataStore; `SavedStateHandle` — не путать с постоянным хранилищем. ⁵⁶

Вопрос

Как выбирать между Room, DataStore и `SavedStateHandle` ?

Junior answer.

Room — для таблиц и сложных данных, DataStore — для небольших настроек, `SavedStateHandle` — для восстановления состояния экрана. Это разные уровни хранения. ⁵⁷

Middle answer.

Если данные должны переживать процесс и быть source of truth приложения — это Room или DataStore, в зависимости от формы и сложности данных. `SavedStateHandle` нужен для небольшого UI-related state, достаточного, чтобы после process death повторно собрать screen state из data layer, а не хранить всё содержимое экрана. ⁵⁸

Senior answer.

Senior отвечает через lifespan и reconstruction cost. Если значение — часть durable business data, оно живёт на диске; если это navigation/filter/query key, достаточный для повторной загрузки и восстановления user journey, его место в `SavedStateHandle`; если это purely ephemeral Compose-local detail, иногда достаточно `rememberSaveable` / `rememberSerializable` без протаскивания в ViewModel. ⁵⁹

Why this question matters.

В production именно путаница между этими уровнями даёт либо избыточное хранение, либо потерю состояния после process death, либо неадекватно раздутые ViewModel. Ответ показывает, умеет ли человек мыслить сроком жизни данных, а не только API. ⁶⁰

Вопрос

Где проходит граница между auto migration и manual migration в Room?

Junior answer.

Auto migration подходит для простых изменений схемы, а сложные случаи лучше делать вручную. Нужно следить за совместимостью. ⁶¹

Middle answer.

Как только изменение перестаёт быть тривиальным rename/add/drop и требует реальной трансформации данных, контроля над SQL или нестандартной логики, manual migration безопаснее и понятнее. Senior-level hygiene здесь — ещё и тестировать migrations, а не верить, что схема “очевидная”. ⁶²

Senior answer.

Хороший senior-ответ включает operational thinking: миграция — это не только схема, но и пользовательские данные, размер таблиц, идемпотентность, rollback story и observability релиза. Если вы не можете уверенно описать, что случится с содержимым базы на реальном устройстве после обновления, значит автоматизация уже не покрывает ваш риск-профиль и нужна ручная миграция с тестом. ⁶²

Why this question matters.

Этот вопрос проверяет, работал ли человек с upgrade path настоящих пользователей, а не только с clean install. Уровень senior часто виден именно в том, насколько он думает про уже существующие данные, а не только про новую схему. ⁶¹

Retrofit и OkHttp

Theory block.

Senior-понимание networking на Android — это не “создал Retrofit interface”, а контроль над transport semantics, caching, error surface и orchestration. Retrofit умеет представлять API как interface; для Kotlin он поддерживает suspend -методы, которые либо возвращают Response<T>, либо сразу body и в случае non-2xx бросают HttpException. Важно не смешивать модели: suspend fun с возвращаемым Call<T> в Retrofit помечается как некорректная комбинация. OkHttp остаётся фундаментом: он по умолчанию эффективен за счёт HTTP/2, connection pooling, transparent GZIP и response cache. Но cache работает не “магически”: документация отдельно предупреждает, что ответ должен быть прочитан полностью, иначе он не закешируется; также разумно держать один shared OkHttpClient, а не плодить клиентов и кэши на одно и то же. Interceptors — мощный, но опасный инструмент: application и network interceptors имеют разную семантику, порядок важен, а proceed() должен использоваться корректно. Senior-дисциплина — разделять transport errors, protocol errors, domain errors и UI errors, не превращая repository в смесь логгера, retry-машины и parser-a. ⁶³

Вопрос

Когда API метод должен возвращать T, а когда Response<T> ?

Junior answer.

Если нужен только body, можно вернуть T. Если нужно читать код ответа и заголовки, лучше Response<T>. ⁶⁴

Middle answer.

Для большинства happy-path endpoint-ов `T` даёт чище API и автоматически переводит non-2xx в `HttpException`. `Response<T>` нужен там, где transport metadata — часть бизнес-решения: например, conditional requests, pagination headers, partial success semantics или необходимость различать empty body и отсутствие ответа. ⁶⁴

Senior answer.

Senior выбирает тип возврата на границе data layer как часть контракта, а не на вкус. Если наружу постоянно течёт `Response<T>`, вы поднимаете transport concerns слишком высоко; если всегда скрываете его, можно потерять важную протокольную семантику. Правильный выбор зависит от того, должен ли вышестоящий слой принимать решение на основе HTTP-метаданных или нет. ⁶⁵

Why this question matters.

Вопрос быстро вскрывает, понимает ли инженер, что networking API — это boundary design, а не просто способ достать JSON. От этого зависит чистота repository contracts и то, насколько протокол протекает в UI/domain. ⁶⁴

Вопрос

Чем application interceptor отличается от network interceptor и где часто ошибаются?

Junior answer.

Они работают на разных уровнях запроса. Application interceptor обычно выше, network interceptor ближе к сети. ⁶⁶

Middle answer.

Application interceptor видит полный span вызова и удобен для cross-cutting concerns вроде auth/header decoration или общих policy-решений. Network interceptor работает ближе к конкретному сетевому обмену и требует большей аккуратности; документация подчёркивает, что `proceed()` надо вызывать корректно, а порядок interceptors влияет на итоговое поведение. ⁶⁶

Senior answer.

Senior смотрит на интерсептор как на транспортный middleware, а не на место “куда бы засунуть побольше логики”. Если retry, metrics, auth-refresh, caching override и request rewriting смешаны без явной policy order, вы теряете наблюдаемость и предсказуемость. Особенно опасно превращать network layer в implicit state machine, которую уже нельзя нормально тестировать или reason about при ошибках. ⁶⁷

Why this question matters.

На боевых приложениях сетевой стек почти всегда со временем обрастает cross-cutting логикой. Этот вопрос показывает, умеет ли человек держать transport pipeline чистым и управляемым, а не только добавлять interceptors по мере появления новой задачи. ⁶⁸

Navigation, lifecycle и resilience

Custom Fragment-based navigation и back stack

Theory block.

Если вы сознательно живёте с custom Fragment-based navigation в 2026, senior-уровень здесь

определяется не знанием `FragmentManager`, а способностью владеть **back stack как данными системы**. `FragmentManager` управляет стеком транзакций и позволяет сохранять/восстанавливать состояние fragment-based navigation; `FragmentManager` доставляется только когда слушающий fragment снова становится `STARTED`, что удобно для возврата результатов через pop. При hosting Compose в Fragment официальные interop-доки требуют аккуратной integration strategy: `ComposeView` во Fragment должен использовать корректную `ViewCompositionStrategy`, обычно `DisposeOnViewTreeLifecycleDestroyed`. Если на одном layout несколько `ComposeView`, им нужны уникальные id для корректной работы `savedInstanceState`. Для back-навигации Android рекомендует `OnBackPressedDispatcher`, а predictive back требует корректной интеграции с современными AndroidX API. При этом в 2026 уже существует стабильный Navigation 3, построенный вокруг Compose state и собственного back stack, так что custom navigation нужно уметь защищать аргументами: модульные границы, control over lifetime, интеграция с существующим Fragment stack, собственные transition/result contracts. Если таких аргументов нет — возможно, вы просто платите infra-tax без выгоды. ⁶⁹

Вопрос

Какие типичные ошибки допускают при hosting Compose screen внутри Fragment?

Junior answer.

Нужно использовать `ComposeView` и правильно вызывать `setContent()`. Ещё важно не забыть про lifecycle. ⁷⁰

Middle answer.

Частые ошибки: неверная disposal strategy для Composition во Fragment, забытый `DisposeOnViewTreeLifecycleDestroyed`, а также отсутствие уникальных id у нескольких `ComposeView`, из-за чего ломается state restoration. Ещё легко забыть, что binding во Fragment живёт только между `onCreateView()` и `onDestroyView()`. ⁷¹

Senior answer.

Senior видит здесь пересечение двух lifecycle world-ов: Fragment view lifecycle и Composition lifecycle. Если screen state или side-effects привязаны не к тому owner-у, можно получить либо преждевременную disposal, либо удержание ресурсов после уничтожения view, либо расхождение между back stack entry и тем, что по факту ещё живёт в памяти. ⁷²

Why this question matters.

Это практический вопрос на реальные hybrid-приложения. Он хорошо показывает, работал ли человек с живой interop-схемой, а не только с Compose-only demo apps. ⁷³

Вопрос

Когда custom navigation действительно оправдана по сравнению с Navigation Compose или Navigation 3?

Junior answer.

Когда уже есть свой стек или нужны нестандартные сценарии. В остальных случаях стандартная библиотека часто проще. ⁷⁴

Middle answer.

Custom stack оправдан, если у вас уже есть Fragment-based host architecture, feature isolation,

собственные result/transition contracts или требования к back stack, которые стандартная библиотека не покрывает без сильных компромиссов. Но это должно окупаться, потому что Navigation 3 уже даёт state-driven back stack для Compose. ⁷⁵

Senior answer.

Senior оценивает не только функциональность, но и platform fit: кто будет поддерживать deep links, predictive back, saved navigation state, modular boundaries и инструменты тестирования через год-два. Если custom nav не даёт явного стратегического преимущества в ваших lifetime/boundary contracts, она становится дорогой внутренней платформой, которую команда фактически пишет и поддерживает вместо продукта. ⁷⁶

Why this question matters.

Вопрос проверяет architectural judgment. Отличие senior здесь в том, умеет ли он обосновать инфраструктурную сложность через долгосрочную выгоду, а не через привычку команды. ⁷⁷

Process lifecycle, WorkManager и process death

Theory block.

Сильный Android-инженер всегда помнит: process не является надёжным контейнером состояния. Android docs прямо описывают, что процесс приложения может быть уничтожен системой в фоне, а `ViewModel` переживает configuration change, но не system-initiated process death. Поэтому senior-стратегия восстановления строится так: сохранить **минимум** transient UI state (`SavedStateHandle`, `rememberSaveable`, в 2026 — и `rememberSerializable` /serialization support там, где уместно), а screen state целиком повторно произвести из data layer. `SavedStateHandle` хорош для query/filter/id-ключей и прочих small UI elements, достаточных для восстановления user journey; но сложный screen state Android docs не рекомендуют складировать туда. Для гарантированной фоновой работы нужен WorkManager: он хранит работу во внутренней SQLite БД, переживает перезагрузки устройства, поддерживает unique work, constraints и expedited work для важных задач. `ProcessLifecycleOwner` полезен как coarse-grained сигнал foreground/background на процесс, но не заменяет screen lifecycle. Senior-уровень здесь — перестать надеяться на “память процесса” как на backend и строить экран так, будто его всегда могут уничтожить и поднять заново. ⁷⁸

Вопрос

Что именно вы сохраняете в `SavedStateHandle`, а что сознательно не сохраняете?

Junior answer.

Сохраняю небольшие значения вроде query, selected tab или id. Большие данные лучше не хранить там. ⁷⁹

Middle answer.

В `SavedStateHandle` стоит сохранять минимальный набор, который нужен, чтобы повторно восстановить экран после process death. Полный screen state, списки сущностей и тяжёлые модели лучше снова произвести из repository/Room/DataStore, иначе handle превращается в хрупкий кэш. ⁸⁰

Senior answer.

Senior отвечает через reconstruction boundary: в `SavedStateHandle` идут ключи восстановления, а не производные данные. Если нельзя чётко объяснить, почему это значение

не может быть снова вычислено из data layer, его, скорее всего, и не нужно сохранять в handle.

81

Why this question matters.

Это вопрос на трезвое мышление про lifespans. Он очень быстро выявляет, кто реально проектирует под process death, а кто до сих пор неявно предполагает “приложение просто висит в памяти”. 82

Вопрос

Когда нужен WorkManager, а когда его использование — оверкилл?

Junior answer.

WorkManager нужен для надёжной фоновой работы, которая должна выполняться даже позже. Для простой работы внутри открытого экрана он может быть лишним. 83

Middle answer.

Если задача должна переживать выход из приложения, ограничения системы и даже reboot, WorkManager — правильный выбор. Если операция напрямую привязана к живому экрану и должна отмениться вместе с ним, её естественное место в screen scope, а не в persistent scheduler. 83

Senior answer.

Senior также смотрит на идемпотентность и ownership: WorkManager хорош для sync/upload/reconcile задач с durable intent и понятной retry policy. Он плох как замена нормальной orchestration-модели UI, когда разработчик просто не хочет разбираться в scopes и жизненном цикле и потому “запускает всё через воркер”. 84

Why this question matters.

Вопрос проверяет, понимает ли инженер различие между screen-bound concurrency и durable background execution. На production это влияет и на UX, и на battery/use-of-resources, и на корректность повторных запусков. 85

Testing и build engineering

Testing strategy

Theory block.

Senior Android testing в 2026 — это не “покрыть проценты”, а построить дешёвую и устойчивую сеть обратной связи. Базовый слой — pure tests reducers/use cases/transformers; выше — actor/repository tests с fake dependencies; ещё выше — Compose UI tests на пользовательские сценарии; вокруг этого — узкий слой integration tests, где проверяются boundary contracts. Для coroutine-кода Android docs рекомендуют `runTest`; он даёт `TestScope`, виртуальное время и контролируемое выполнение child coroutines. Для Flow-тестов важно различать, является ли поток входом или выходом системы под тестом: тестируются либо fake producers, либо сбор эмиссий subject under test. `backgroundScope` полезен для долговременных collectors, которые должны автоматически отмениться в конце теста. В Compose testing в 2026 существует переход на `runComposeUiTest`: он сам выполняет тест внутри `runTest` и синхронизирует test clock с Compose окружением, что уменьшает ручное управление scheduler-ом. В hybrid UI Compose и Views можно тестировать одновременно: Espresso для View и ComposeTestRule для composables.

JUnit Jupiter ценен extension model-ю и гибкостью, а MockK — удобной поддержкой Kotlin/coroutines, но senior не подменяет дизайн тестируемости магией моков. ⁸⁶

Вопрос

Почему coroutine-тесты остаются flaky даже с `runTest`?

Junior answer.

Потому что не всё управление временем и dispatchers настроено правильно. Иногда корутины стартуют вне test scope. ⁸⁷

Middle answer.

Типичные причины — hardcoded dispatchers, фоновые collectors вне `TestScope`, race между запуском и ассертами, а также код, который живёт дольше теста. `runTest` помогает, но не исправляет архитектуру автоматически. ⁸⁸

Senior answer.

Senior отвечает системно: flaky-тест обычно сигнализирует не о “плохом `runTest`”, а о том, что execution ownership в production-коде неявен. Если объект под тестом сам создаёт scope/dispatcher, держит скрытый background job или смешивает горячие стримы без явного lifecycle, тест просто честно показывает архитектурную неуправляемость. ⁸⁹

Why this question matters.

Вопрос проверяет, видит ли человек тест как диагностику дизайна, а не как обряд с `advanceUntilIdle()`. Это критично для команд, где flaky suite медленно, но верно убивает доверие к CI. ⁹⁰

Вопрос

Как вы тестируете `StateFlow` / `SharedFlow` output экрана или feature?

Junior answer.

Подписываюсь на поток и проверяю эмиссии. Для hot stream важно успеть начать collection до события. ⁹¹

Middle answer.

Для `StateFlow` обычно проверяется стартовое состояние и последовательность переходов после intents. Для `SharedFlow` /events нужен долговременный collector в test scope, иначе transient emission можно пропустить; Android-гайд по Flow-test явно разделяет подходы к flow-as-input и flow-as-output. ⁹²

Senior answer.

Senior тестирует не “что что-то эмитнулось”, а контракт state machine: какие intents порождают какие states и какие one-off events при каком состоянии мира. Хороший тестовый дизайн ещё и минимизирует knowledge о внутренних операторах, чтобы refactor pipeline не ломал тесты, пока сохраняется внешний реактивный контракт. ⁹³

Why this question matters.

Это вопрос на умение тестировать реактивную систему по её observable contract. Он отлично

отделяет людей, которые проверяют случайные implementation details, от людей, которые защищают инварианты экрана. ⁹⁴

Вопрос

Что должна проверять Compose UI test, чтобы оставаться полезной и не стать хрупкой?

Junior answer.

Нужно проверять пользовательский сценарий: видимость элементов, взаимодействия, текст, переходы состояния. Не стоит слишком завязываться на внутреннюю реализацию. ⁹⁵

Middle answer.

Хороший Compose UI test опирается на semantics и observable user outcome, а не на внутреннюю структуру composable tree. В hybrid-экранах нужно естественно сочетать Compose и Espresso, а не пытаться одной системой тестировать всё подряд не по месту. ⁹⁶

Senior answer.

Senior обычно держит UI tests узкими и ценными: smoke на critical journeys, а сложную state logic тестирует ниже. Идея в том, чтобы UI test подтверждал интеграцию rendering + input + async synchronization, а не дублировал десятки reducer/unit tests в более медленной и дорогой форме. ⁹⁷

Why this question matters.

В production UI tests либо становятся активом, либо превращаются в дорогой шум. По ответу видно, умеет ли человек строить тестовую пирамиду, где каждый слой делает свою работу. ⁹⁸

Build, modularization и performance

Theory block.

Сильный senior Android developer рассматривает build и performance как часть feature-delivery, а не как постфактум-оптимизацию. Version Catalogs централизуют dependency management и хорошо сочетаются с multi-module структурой. Но modularization, по Android docs, не имеет единственного правильного шаблона: модульные границы должны соответствовать ownership, сборочным скоростям, API/implementation surface и способности команды эволюционировать систему. Для навигации и feature boundaries в 2026 Android отдельно обсуждает разделение на `api` и `impl` submodules. R8 — это не только shrinking, но и оптимизация/обфускация; Android docs отдельно предупреждают не превращать релиз в debug-сборку глобальными `-dontshrink`, `-dontobfuscate` и `-dontoptimize`. Baseline Profiles дают ощутимое ускорение кода уже с первого запуска, а Startup Profiles дополнительно оптимизируют DEX layout и ускоряют старт ещё сильнее. Но порядок мышления важен: перформанс надо измерять; Compose stability fixes и strong skipping применяются после диагностики, а не как ритуал. Senior-уровень тут — связывать архитектурные границы, build cost, cold start и runtime responsiveness в одну систему решений. ⁹⁹

Вопрос

Как выбирать module boundaries, чтобы они были полезны, а не декоративны?

Junior answer.

Нужно делить проект на логические части, например по feature-ам или слоям. Так проще поддерживать код. ¹⁰⁰

Middle answer.

Полезная граница уменьшает compilation impact, делает API surface явным и закрепляет ownership. Если модуль существует, но всё равно тянет половину проекта через `api` или требует повсеместных friend-like знаний, то модульность декоративна. ¹⁰¹

Senior answer.

Senior проектирует модули вокруг change-rate и команды, а не только вокруг папок. Хорошая модульность облегчает parallel development, снижает blast radius изменений и даёт контракты, которые можно тестировать и оптимизировать независимо; плохая — увеличивает Gradle overhead и DI/navigation complexity без реального снижения связанности. ¹⁰¹

Why this question matters.

Этот вопрос показывает, понимает ли инженер modularization как инструмент управления сложностью, а не как визуальную организацию репозитория. На scale это напрямую влияет на скорость команды и стоимость изменений. ¹⁰⁰

Вопрос

Куда инвестировать первым делом: Baseline Profiles, Startup Profiles, R8 tuning или Compose stability work?

Junior answer.

Сначала надо померить проблему, а потом оптимизировать. Часто Baseline Profiles и R8 дают заметный эффект. ¹⁰²

Middle answer.

Если страдает cold start и critical journeys, Baseline Profiles часто дают быстрый и измеримый выигрыш; Startup Profiles усиливают startup-часть ещё сильнее. Compose stability work нужен, когда трассировка показывает лишнюю recomposition/phase work, а R8 tuning — когда видно, что размер и optimizer policy реально влияют на запуск и runtime. ¹⁰³

Senior answer.

Senior отвечает не списком фич, а последовательностью: instrumentation → hypothesis → smallest high-leverage change → re-measure. Baseline/Startup Profiles и R8 улучшают системные характеристики приложения в целом, тогда как stability fixes часто более локальны; но если hotspot — конкретный Compose screen под скроллом, глобальные build-оптимизации не заменят точечной работы с invalidation model. ¹⁰⁴

Why this question matters.

Вопрос проверяет инженерное мышление о performance как о measured system, а не о наборе модных практик. Senior почти всегда отличается тем, что умеет расставлять оптимизации по ROI, а не по хайпу. ¹⁰⁵

Практические задачи

Task для coroutines и MVI orchestration

Реализуйте `Actor`, который по intent `Refresh(force: Boolean)` параллельно запускает два запроса: `loadProfile()` и `loadPermissions()`. Если `profile` упал, экран должен перейти в `Error`; если `permissions` упали, экран должен показать `News.PermissionWarning`, но остаться в `Content`.

Что проверяет. Structured concurrency, выбор между `coroutineScope` и `supervisorScope`, границы `Actor` / `Reducer` / `NewsPublisher`, fault model экрана. ¹⁰⁶

Task для Flow и hot stream semantics

Дан repository, который предоставляет cold `Flow<List<Item>>`. Превратите его в screen-level `StateFlow<ItemsState>` так, чтобы expensive upstream не перезапускался на каждый новый collector, но корректно останавливался после ухода экрана. Объясните выбор `stateIn` и `SharingStarted`.

Что проверяет. Cold vs hot, `stateIn`, ownership of scope, replay semantics, восстановление подписки после возврата на экран. ¹⁰⁷

Task для Compose performance

Есть `LazyColumn` со списком из 5 000 элементов. Внутри каждого item есть локальный remembered state, а сортировка списка меняется по нажатию кнопки. Исправьте код так, чтобы при reorder remembered state не “переезжал”, а recomposition footprint был минимальным.

Что проверяет. Stable keys, state locality, invalidation regions, базовое профилирование горячего списка. ³⁷

Task для Fragment-hosted Compose migration

Напишите `Fragment`, который хостит целиком Compose screen через `ComposeView`, корректно освобождает Composition вместе с `viewLifecycleOwner` и переживает `savedInstanceState`. Дополнительно покажите, как безопасно вернуть результат назад в предыдущий Fragment.

Что проверяет. Compose/View interop,

`ViewCompositionStrategy.DisposeOnViewTreeLifecycleDestroyed`, back stack semantics, fragment result delivery. ¹⁰⁸

Task для persistence layer

Смоделируйте экран фильтрации каталога, где: выбранные toggles и sort order должны переживать process death, а список товаров должен пересобираться из локального source of truth. Разделите, что хранится в Room, что в DataStore, а что в `SavedStateHandle`.

Что проверяет. Lifespans данных, source of truth, reconstruction after process death, практическая граница Room/DataStore/SavedStateHandle. ¹⁰⁹

Task для testing strategy

Напишите тест для feature/ViewModel, который при старте собирает repository flow, а по событию `Retry` переходит `Loading -> Content` или `Loading -> Error`. Тест должен быть детерминированным, использовать `runTest`, не течь корутинами после завершения и

отдельно проверять transient event.

Что проверяет. `runTest`, `TestScope`, continuous collection, тестирование `StateFlow` и one-off event contracts, дизайн тестов без лишней зависимости от implementation details. ¹¹⁰

Как использовать эту базу для self-calibration

Если вы отвечаете на вопросы в стиле Junior/Middle, это не значит, что у вас “плохая база”; чаще это означает, что вам не хватает опыта работы с **контрактами системы под нагрузкой времени**: process death, hot streams, replay semantics, scope ownership, DI graph boundaries, migration cost, performance regression и тестовой детерминированности. Сильный senior в Android 2026 почти всегда мыслит через lifetime, ownership, replay, observability и cost of change. Именно эти оси объединяют seemingly разные темы — coroutines, Compose, DI, storage, navigation, testing и build engineering — в одну инженерную систему. ¹¹¹

Практически это означает следующее. Если вы хотите расти именно к senior-калибру под указанный стек, полезно перепроверять любой свой design choice одним и тем же набором вопросов: кто владеет жизненным циклом; что произойдёт при повторной подписке; что случится после process death; какой здесь durable state, а какой transient event; как это тестируется детерминированно; как это деградирует по производительности; какова цена миграции и поддержки через полгода. Если на эти вопросы есть чёткие ответы — архитектура обычно здорова. Если нет, проблема чаще всего не в API, а в неявных границах ответственности. ¹¹²

¹ Guide to app architecture

https://developer.android.com/topic/architecture?utm_source=chatgpt.com

² ¹¹² Recommendations for Android architecture

https://developer.android.com/topic/architecture/recommendations?utm_source=chatgpt.com

³ Coroutines | Kotlin Documentation

https://kotlinlang.org/docs/coroutines-overview.html?utm_source=chatgpt.com

⁴ ⁵ ⁶ ⁷ ¹⁰⁶ Coroutine exceptions handling

https://kotlinlang.org/docs/exception-handling.html?utm_source=chatgpt.com

⁸ ⁹ ¹⁰ CoroutineExceptionHandler | kotlinx.coroutines

https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-core/kotlinx.coroutines/-coroutine-exception-handler/?utm_source=chatgpt.com

¹¹ Use Kotlin coroutines with lifecycle-aware components

https://developer.android.com/topic/libraries/architecture/coroutines?utm_source=chatgpt.com

¹² ¹³ ¹⁴ ⁸⁶ ⁸⁷ ⁸⁸ ⁸⁹ ⁹⁰ ¹¹⁰ Testing Kotlin coroutines on Android

https://developer.android.com/kotlin/coroutines/test?utm_source=chatgpt.com

¹⁵ Flow | kotlinx.coroutines

https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-core/kotlinx.coroutines.flow/-flow/?utm_source=chatgpt.com

¹⁶ ¹⁷ ¹⁸ StateFlow | kotlinx.coroutines

https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-core/kotlinx.coroutines.flow/-state-flow/?utm_source=chatgpt.com

¹⁹ ⁴⁰ ⁴¹ ⁴² ⁴³ Events that should only be consumed only once - MVICore

<https://badoo.github.io/MVICore/features/news/>

20 21 107 **stateIn | kotlinx.coroutines**

https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-core/kotlinx.coroutines.flow/state-in.html?utm_source=chatgpt.com

22 **shareIn | kotlinx.coroutines**

https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-core/kotlinx.coroutines.flow/share-in.html?utm_source=chatgpt.com

23 24 25 26 **kotlinx.coroutines-rx2**

https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-rx2/?utm_source=chatgpt.com

27 28 **Stability in Compose | Jetpack Compose**

https://developer.android.com/develop/ui/compose/performance/stability?utm_source=chatgpt.com

29 31 **Diagnose stability issues | Jetpack Compose**

https://developer.android.com/develop/ui/compose/performance/stability/diagnose?utm_source=chatgpt.com

30 **Follow best practices | Jetpack Compose**

https://developer.android.com/develop/ui/compose/performance/bestpractices?utm_source=chatgpt.com

32 33 34 **Side-effects in Compose**

https://developer.android.com/develop/ui/compose/side-effects?utm_source=chatgpt.com

35 36 37 38 **Lazy lists and lazy grids | Jetpack Compose**

https://developer.android.com/develop/ui/compose/lists?utm_source=chatgpt.com

39 47 **UI State production | App architecture**

https://developer.android.com/topic/architecture/ui-layer/state-production?utm_source=chatgpt.com

44 45 46 **Handling async jobs - MVICore**

https://badoo.github.io/MVICore/features/actorreducerfeature/?utm_source=chatgpt.com

48 53 **Dagger**

https://dagger.dev/dev-guide/?utm_source=chatgpt.com

49 **Subcomponents**

https://dagger.dev/dev-guide/subcomponents.html?utm_source=chatgpt.com

50 **Subcomponents vs Component dependencies**

https://dagger.dev/hilt/subcomponents-vs-deps.html?utm_source=chatgpt.com

51 100 101 **Guide to Android app modularization | App architecture**

https://developer.android.com/topic/modularization?utm_source=chatgpt.com

52 54 **Hilt**

https://dagger.dev/hilt/?utm_source=chatgpt.com

55 **Dependency injection in Android | App architecture**

https://developer.android.com/training/dependency-injection?utm_source=chatgpt.com

56 57 **Save data in a local database using Room**

https://developer.android.com/training/data-storage/room?utm_source=chatgpt.com

58 59 79 80 **Saved State module for ViewModel | App architecture**

https://developer.android.com/topic/libraries/architecture/viewmodel/viewmodel-savedstate?utm_source=chatgpt.com

60 **Save UI states | App architecture**

https://developer.android.com/topic/libraries/architecture/saving-states?utm_source=chatgpt.com

61 62 **Migrate your Room database | App data and files**

https://developer.android.com/training/data-storage/room/migrating-db-versions?utm_source=chatgpt.com

63 **Introduction | Retrofit**

https://square.github.io/retrofit/?utm_source=chatgpt.com

64 65 **Declarations | Retrofit**

https://square.github.io/retrofit/declarations/?utm_source=chatgpt.com

66 67 68 **Interceptors - OkHttp**

https://square.github.io/okhttp/features/interceptors/?utm_source=chatgpt.com

69 **Fragment manager | App architecture**

https://developer.android.com/guide/fragments/fragmentmanager?utm_source=chatgpt.com

70 71 72 73 108 **Using Compose in Views | Jetpack Compose | Android Developers**

<https://developer.android.com/develop/ui/compose/migrate/interoperability-apis/compose-in-views>

74 75 77 **Jetpack Navigation 3 is stable**

https://developer.android.com/blog/posts/jetpack-navigation-3-is-stable?utm_source=chatgpt.com

76 **Add support for the predictive back gesture | App architecture**

https://developer.android.com/guide/navigation/custom-back/predictive-back-gesture?utm_source=chatgpt.com

78 82 111 **Processes and app lifecycle | App architecture**

https://developer.android.com/guide/components/activities/process-lifecycle?utm_source=chatgpt.com

81 **State holders and UI state | App architecture**

https://developer.android.com/topic/architecture/ui-layer/stateholders?utm_source=chatgpt.com

83 85 **Task scheduling | Background work**

https://developer.android.com/develop/background-work/background-tasks/persistent?utm_source=chatgpt.com

84 **Build an offline-first app | App architecture**

https://developer.android.com/topic/architecture/data-layer/offline-first?utm_source=chatgpt.com

91 92 93 94 **Testing Kotlin flows on Android**

https://developer.android.com/kotlin/flow/test?utm_source=chatgpt.com

95 97 **Migrate to v2 testing APIs | Jetpack Compose**

https://developer.android.com/develop/ui/compose/testing/migrate-v2?utm_source=chatgpt.com

96 **Interoperability | Jetpack Compose**

https://developer.android.com/develop/ui/compose/testing/interoperability?utm_source=chatgpt.com

98 **Overview :: JUnit User Guide**

https://docs.junit.org/6.1.1/overview.html?utm_source=chatgpt.com

99 **Version Catalogs**

https://docs.gradle.org/current/userguide/version_catalogs.html?utm_source=chatgpt.com

102 103 **Baseline Profiles overview | App quality**

https://developer.android.com/topic/performance/baselineprofiles/overview?utm_source=chatgpt.com

104 105 **Benchmark Baseline Profiles with Macrobenchmark library**

https://developer.android.com/topic/performance/baselineprofiles/measure-baselineprofile?utm_source=chatgpt.com

109 **App Architecture: Data Layer - DataStore**

https://developer.android.com/topic/libraries/architecture/datastore?utm_source=chatgpt.com